
Instruct-3D-to-3D: Editing 3D Scenes with Instructions

Eliza Huang
heliza@stanford.edu

Fan Xie
xiefan46@stanford.edu

Chi Yo Tsai
chiyo@stanford.edu

1 Introduction

With recent advances in pre-trained large-scale models, the rapid generation of 3D content from scratch has become feasible. This can be achieved either by optimizing radiance fields through vision-language models like CLIP or by employing text-conditioned diffusion models, exemplified by DreamFusion [7]. Neural Radiance Fields (NeRFs) have emerged as a potent representation, showcasing an unparalleled ability to encode scenes with exceptional fidelity [5]. However, the challenge lies in the editing of NeRFs, primarily due to the intricacies of their underlying representation. Rather than focusing on generating new content, our emphasis is on editing NeRFs derived from real, fully observed scenes, leveraging 2D diffusion priors. A key hurdle is addressing the inconsistent outputs of a 2D diffusion model when translated into a cohesive 3D scene. This project explores an approach to editing 3D scenes based on intuitive, language-based editing instructions. Natural-language instructions provide an intuitive and user-friendly interface, allowing even novice users to achieve high-quality results without the need for additional tools or specialized knowledge.

The codebase is available on <https://github.com/lizzij/instruct-3d-to-3d>.

2 Related Work

2.1 Zero-1-to-3

Zero-1-to-3 serves as a framework designed to alter the camera perspective of an object, relying solely on a single RGB image. Operating within the challenging realm of novel view synthesis, where constraints are abundant, Zero-1-to-3 strategically harnesses the geometric priors ingrained in large-scale diffusion models' understanding of natural images. Through a conditional diffusion model trained on a synthetic dataset, the system learns controls over the relative camera viewpoint. These controls facilitate the generation of new images of the same object under specified camera transformations. Despite being trained on synthetic data, the model showcases robust zero-shot generalization capabilities, extending its efficacy to out-of-distribution datasets and diverse in-the-wild images, including those portraying impressionist paintings. The viewpoint-conditioned diffusion approach is further applicable to the task of 3D reconstruction from a single image. Rigorous qualitative and quantitative experiments underscore the method's superiority over state-of-the-art single-view 3D reconstruction and novel view synthesis models, owing to its leverage of Internet-scale pre-training [4].

2.2 InstructPix2Pix

InstructPix2Pix introduces an approach for image editing guided by human instructions. When presented with an input image and a corresponding written directive, our model adeptly adheres to these instructions to perform the desired image edits. Acquiring training data for this nuanced task involves leveraging the expertise of two extensively pretrained models—a language model (GPT-3) and a text-to-image model (Stable Diffusion). Through this fusion of knowledge, InstructPix2Pix generates an extensive dataset of image editing examples. The conditional diffusion model, InstructPix2Pix, is

subsequently trained on this generated data and demonstrates impressive generalization to real images and user-provided instructions during inference. Notably, the model’s capacity to execute edits in the forward pass obviates the need for per-example fine-tuning or inversion, resulting in rapid image editing within seconds. InstructPix2Pix showcases compelling editing outcomes across a diverse array of input images and associated written instructions [1].

2.3 NeRF

Neural Radiance Field (NeRF) is a way of representing a 3D scene with a deep neural network. The key insight behind this representation is that the rendering function that renders an image from a given NeRF and a camera pose is differentiable, so back-propagation can be used to optimize for a scene that’s consistent with a given set of views [5].

2.4 DreamFusion

DreamFusion is one of the first successful generative model for 3D scene generation [7]. It relies on using a NeRF for scene representation, and use a score-based image model to evaluate the quality of rendering from the NeRF. The authors then use the image model’s score as a proxy to "back-propagate" and optimize for the best NeRF.

2.5 Instruct 3D to 3D by Sony

The paper in [3] also performs 3D scene editing using an approach similar to ours. The main difference being that their formulation is based on DreamFusion, whereas ours is based on Score Jacobian Chaining.

3 Technical Approach

Consider a 3D scene $\mathbf{v}$ represented by a NeRF-like model θ_o that we can use to render images from different camera intrinsic parameters and different extrinsic poses. For simplicity, we denote the combination of camera and pose by P and the rendering $\mathbf{x}(\theta_o, P)$. Given an instruction prompt T to edit the scene, our goal is to produce a new 3D scene parameterized by θ that is consistent with the original scene $\mathbf{v}$ and the text instruction T . We denote the conditional variables (i.e. the original scene and the text instruction) by y . We aim to produce new scene θ by implicitly maximizing the log-likelihood of the 3D model $p(\theta|y)$.

3.1 Score Jacobian Chaining

Currently, there is no well-known generative models for 3D scene generation due to lack of large-scale dataset for 3D scenes. However, there are many successful image generative models. In particular, Latent Diffusion Models (LDM) have recently been very successful in generating high quality images. Our approach is to leverage these image models to approximate the log-likelihood of 3D scenes $p(\theta; y)$, and perform back propagation to find the locally optimal θ . This approach is known as Score Jacobian Chaining [9].

As we are using LDM in our approach, we will denote the noisy log-likelihood with noise level σ by p_σ . One particularly important result of these score based models is that, if D_σ is the denoising function at noise level σ , then the score of noisy input can be approximated by

$$\nabla_{\mathbf{x}} \log p_\sigma(\mathbf{x}|y) \approx \frac{D_\sigma(\mathbf{x}|y) - \mathbf{x}}{\sigma}$$

To approximate the 3D log-likelihood with image log-likelihood, we assume that the 3D probability is proportional to the image probability average over all camera poses $p_\sigma(\theta|y) = \frac{\mathbb{E}_P[p_\sigma(\mathbf{x}(\theta, P)|y)]}{Z}$, where $Z = \int \mathbb{E}_P[p_\sigma(\mathbf{x}(\theta, P)|y)] d\theta$. So the log-likelihood is lower bounded by

$$\log p_\sigma(\theta|y) = \log(\mathbb{E}_P[p_\sigma(\mathbf{x}(\theta, P)|y)]) - \log Z \geq \mathbb{E}_P[\log p_\sigma(\mathbf{x}(\theta, P)|y)] - \log Z$$

Taking the derivative of the lower bound with respect to θ , we get

$$\nabla_\theta(\mathbb{E}_P[\log p_\sigma(\mathbf{x}(\theta, P)|y)] - \log Z) = \mathbb{E}_P[\nabla_{\mathbf{x}} \log p_\sigma(\mathbf{x}(\theta, P)|y) * \nabla_\theta \mathbf{x}(\theta, P)]$$

This expression for gradient is enough to help us optimize for θ . We could also follow the original SJC paper and introduce some regularization losses for the 3D models (e.g. depth smoothness loss, volume emptiness loss). In practice, we do not observe a significant difference in the quality of generated models, so these were omitted in the final results.

We want to note a couple subtleties in our approach. First, our approach implicitly maximizes a lower bound of the the log-likelihood. However, as typical in a score based formulation, the loss is not tractable to evaluate. Second, because a LDM is used, we do not actually operate in pixel space. To keep the computation in memory, the rendering made from NeRF $\mathbf{x}(\theta, P)$ is actually the representation of the pixel image in latent space. To obtain the renderings in pixel space, we need to perform first-stage decoding. So technically, this variable more accurately denoted by $\mathbf{Z}(\theta, P)$, but we omit the distinction for brevity's sake. Third, in the expression for gradient, the expectation over the poses is usually evaluated with Monte Carlo approximation. In our experiment, a batch size of 1 is sufficient to get good outputs from our model.

In the next two sections, we will specialize this gradient expression for different models.

3.2 Scene Editing with Zero-123

Our naive baseline approach is to directly sample an image from the original scene $\mathbf{v}$, and then use InstructPix2Pix and a prompt T to produce an edited image. Based on this edited image, Zero-1-to-3 can pair it with new camera pose P to construct new views of the edited image. We perform SJC using Zero-123 to obtain the new 3D model.

Zero123 is a conditional LDM where the the conditional variable is a source image concatenated with a camera motion, so $y = [\text{src image}; \text{camera motion}]$. The conditioning is done with classifier free guidance, so $D_\sigma(x|y) = x - \hat{\epsilon}(x|y)$, where $\hat{\epsilon} = \hat{\epsilon}(x|\emptyset) + s(\hat{\epsilon}(x|y) - \hat{\epsilon}(x|\emptyset))$. In our experiment, we keep the guidance strength $s = 100$.

Putting everything together, our gradient is approximated by

$$-\frac{\hat{\epsilon}(\tilde{\mathbf{x}}|\emptyset) + s(\hat{\epsilon}(\tilde{\mathbf{x}}|y) - \hat{\epsilon}(\tilde{\mathbf{x}}|\emptyset))}{\sigma} \nabla_\theta \mathbf{x}(\theta, P) \quad (1)$$

where $\tilde{\mathbf{x}} = \frac{1}{\sqrt{1+\sigma^2}} \mathbf{x}(\theta, P) + \sigma \mathbf{n}$ is the noisy version of the rendered image. See Algorithm 1 for more details.

Algorithm 1 Algorithm for 3D scene editing with Zero-123

Input: 3D scene v , text instruction T , rendering function $\mathbf{r}$, number iterations N_{ite} , regularization weight w , classifier free guidance strength s , learning rate l

Output: A NeRF model parameterized by θ that represents v edited by T

$\theta \leftarrow$ random initialization

$P \leftarrow$ random pose

$I \leftarrow v$ rendered with pose P

$I_{\text{edited}} \leftarrow \text{InstructPix2Pix}(I, T)$

$n \leftarrow 0$

while $n \leq N_{ite}$ **do**

$P' \leftarrow$ randomly sampled pose

$\mathbf{z} \leftarrow \mathbf{r}(\theta, P')$

$M \leftarrow$ motion from P to P'

$y \leftarrow [I_{\text{edited}}; M]$

$\sigma \leftarrow$ sampled from noise schedule

$\mathbf{n} \leftarrow$ sampled from standard Gaussian

$\tilde{\mathbf{z}} \leftarrow \frac{\mathbf{z}}{\sqrt{1+\sigma^2}} + \sigma^2 \mathbf{n}$

$\hat{\epsilon} \leftarrow \hat{\epsilon}(\tilde{\mathbf{z}}|\emptyset) + s(\hat{\epsilon}(\tilde{\mathbf{z}}|y) - \hat{\epsilon}(\tilde{\mathbf{z}}|\emptyset))$

$\triangleright \hat{\epsilon}$ is noise estimate from Zero123

$\theta \leftarrow \theta - l(\frac{\hat{\epsilon}}{\sigma} \nabla_\theta \mathbf{r}(\theta, P))$

$n \leftarrow n + 1$

end while

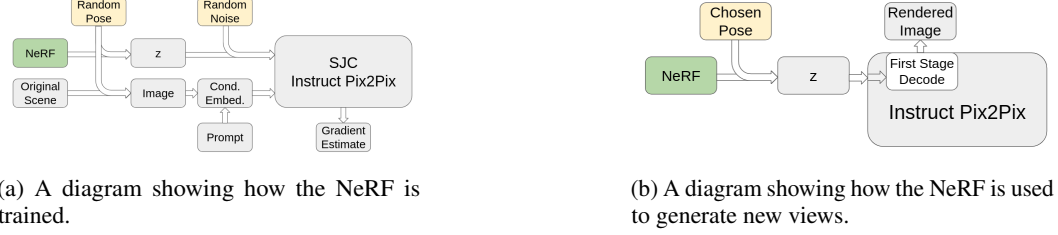


Figure 1: A system diagram of Instruct 3D to 3D.

3.3 Scene Editing with InstructPix2Pix

In this alternative approach, we want to exploit the multiple views that are available from the original 3D model. Therefore, we bypass Zero-123, and try to maximize the log likelihood of the new scene by estimating the gradient with InstructPix2Pix.

InstructPix2Pix is conditioned on two variables, the source image I and the editing instruction T , so the noise estimate is given by

$$\hat{\epsilon} = \hat{\epsilon}(x|\emptyset, \emptyset) + s_{\text{img}} (\hat{\epsilon}(x|I, \emptyset) - \hat{\epsilon}(x|\emptyset, \emptyset)) + s_{\text{txt}} (\hat{\epsilon}(x|I, T) - \hat{\epsilon}(x|I, \emptyset))$$

where s_{img} is the classifier-free guidance strength of image conditioning, and s_{txt} is that of text conditioning. To select the proper conditioning strength, we start by rendering a single image from the original 3D scene, and manually tune the instruction prompt and the strengths until we achieve an aesthetically pleasing output from InstructPix2Pix on that single image.

See Algorithm 2 for more details, and 1 for a system diagram of our approach.

Algorithm 2 Algorithm for 3D scene editing with InstructPix2Pix

Input: 3D scene v , editing prompt T , rendering function $\mathbf{r}$, number iterations N_{ite} , classifier free guidance strength $(s_{\text{img}}, s_{\text{txt}})$, learning rate l

Output: A NeRF model parameterized by θ that represents v edited by T

$\theta \leftarrow$ a NeRF model that represents v well

$n \leftarrow 0$

while $n \leq N_{\text{ite}}$ **do**

$P \leftarrow$ randomly sampled pose

$I \leftarrow \mathbf{r}(v, P)$

$\mathbf{z} \leftarrow \mathbf{r}(\theta, P)$

$\sigma \leftarrow$ sampled from noise schedule

$\mathbf{n} \leftarrow$ sampled from standard Gaussian

$\tilde{\mathbf{z}} \leftarrow \frac{\mathbf{z}}{\sqrt{1+\sigma^2}} + \sigma^2 \mathbf{n}$

$\hat{\epsilon} = \hat{\epsilon}(\tilde{\mathbf{z}}|\emptyset, \emptyset) + s_{\text{img}} (\hat{\epsilon}(\tilde{\mathbf{z}}|I, \emptyset) - \hat{\epsilon}(\tilde{\mathbf{z}}|\emptyset, \emptyset)) + s_{\text{txt}} (\hat{\epsilon}(\tilde{\mathbf{z}}|I, T) - \hat{\epsilon}(\tilde{\mathbf{z}}|I, \emptyset))$

$\theta \leftarrow \theta - l(\frac{\hat{\epsilon}}{\sigma} \nabla_{\theta} \mathbf{r}(\theta, P))$

$n \leftarrow n + 1$

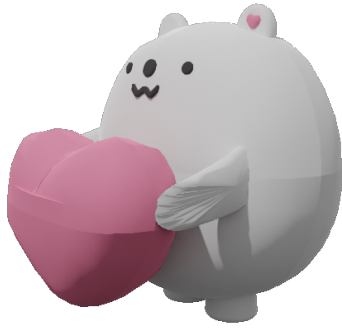
end while

4 Results

4.1 Experimental Procedure

4.1.1 Dataset

We use Objaverse dataset, which is a large-scale open-source dataset containing 800K+ 3D models created by 100K+ artists [2]. Objaverse embodies a large diversity of high-quality 3D models with rich geometry, many of them with fine-grained details and material properties. For our experiment, we only need to train a NeRF through pre-trained image diffusion models, and no further fine-tuning of image models is done. So only a couple samples from Objaverse is needed. We also note that InstructPix2Pix is not trained on rendering of images from 3D models. Consequently, 3D models



(a) Original rendering of a hamster.



(b) Hamster edited with the prompt "Turn the heart into a star."

Figure 2: An example of failure to edit by InstructPix2Pix

with out-of-sample characteristics (e.g. overly smooth textures, less color variations) can lead to editing failures from InstructPix2Pix. See Figure 2 for one such example of failure to edit. As a workaround, we manually select a couple models from Objaverse dataset with complex textures for our experiment. Lee’s Motorworks is one example that we found to work well for a variety of edits.

4.1.2 Training Details

As training a 3D NeRF scene is resource intensive (about 1 hours per trained scene), we tune the parameters for InstructPix2Pix only on a single rendering of the original scene. In all our tests, we set the classifier guidance strength to $s = 100$ for Zero123, and $s_{\text{txt}} = 7.5$, $s_{\text{img}} = 1.5$ for InstructPix2Pix.

For our training procedure, we pre-render 10000 samples of the 3D scene under the same lighting, camera with angle 36.10° , different poses with $z \geq 0$. We use pose batch size of 1 to approximate the gradient, so each sample corresponds to a new iteration in the main training loop. The training rate is set to 0.05. No extra regularization is used as they did not appear to improve the subjective quality.

We use VoxNeRF for our NeRF backend. The voxel grid has 100 voxels in each dimension. The renders from NeRF has dimension of 32 by 32 pixels, and 3 channels.

4.2 Evaluations

4.2.1 Objective Evaluation

To quantitatively evaluate the generated 3D scene, we use CLIP score [8] and BRISQUE score [6]. The CLIP score is a measure of the semantic alignment of image-text pairs, where higher is better. The BRISQUE score is a measure of image quality, where lower is better. Figure 3 shows the results of CLIP score and BRISQUE score of our model compared to the baseline model.

4.2.2 Subjective Evaluation

Figure 4 shows the edits made by InstructPix2Pix to be fed into Zero123. InstructPix2Pix can run into issues where the original details are not preserved in the final output.

Figure 5 shows a comparison of the scenes generated with Zero123 and InstructPix2Pix. We note that the models generated by InstructPix2Pix are more aesthetically pleasing. Zero123 sometimes have issues dealing with generated images, leading to significant blurring and artifacts. The depth map of the generated model is also significantly less accurate. We attribute this to the fact that only a single view was available to Zero123 in the construction of NeRF.



Figure 3: Comparison of CLIP and BRISQUE Scores



(a) "Lee's Autowork" edited with InstructPix2Pix with the prompt "Turn it into lego blocks."



(b) "Lee's Autowork" edited with InstructPix2Pix with the prompt "Turn it into a medieval castle."

Figure 4: "Lee's Autowork" edited by InstructPix2Pix with various prompts to create NeRF scene with Zero123.

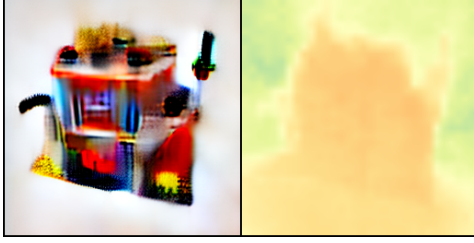
In contrast, using SJC with InstructPix2Pix preserves the depth map and the original structures much better. Nonetheless, our method can run into issues where the object's details got smoothed out (c.f. Figure 5d).

4.3 Analysis

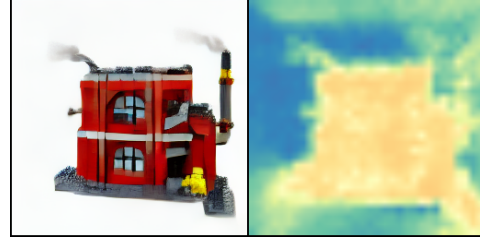
Our method represents the 3D scene with a NeRF model. As such, the best achievable quality is upper-bounded by the capacity of the NeRF model. Unfortunately, due to resource constraints, our NeRF model have limited capacity, so the edited scenes have issues of bluriness and over-smoothing. A better method of upsampling the NeRF output might produce sharper images.

Despite our 3D scene editing method with InstructPix2Pix have worse BRISQUE score than that with Zero123, subjectively we find the output from the former method to be more aesthetically pleasing. While the method using Zero123 can produce good views when the pose is similar to the input views, our method with InstructPix2Pix tends to be more spatially coherent and provides better depth estimates.

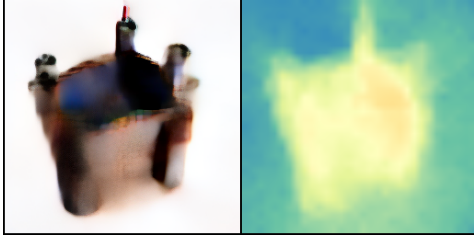
One main issue with our method is that it relies on InstructPix2Pix to perform edits that's consistent with the input models. In practice, we find the actual edits to be very sensitive to various parameters, such as the editing prompt and the classifier guidance strengths. Tuning these various can be expensive computationally, as generating a 3D scene (about an hour) is orders of magnitudes slower than generating a single image (a couple seconds). We also found that InstructPix2Pix can struggle



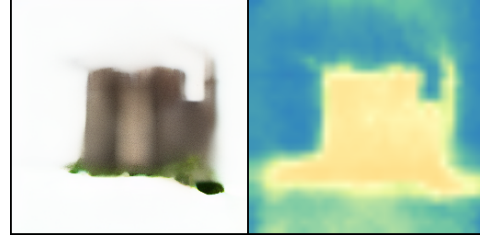
(a) NeRF generated using SJC with Zero123 and the prompt "Turn it into lego blocks."



(b) NeRF generated using SJC with InstructPix2Pix and the prompt "Turn it into lego blocks."



(c) NeRF generated using SJC with Zero123 and the prompt "Turn it into a medieval castle."



(d) NeRF generated with InstructPix2Pix and the prompt "Turn it into a medieval castle."

Figure 5: Comparison of NeRFs generated by Zero123 and InstructPix2Pix.

with some of the renderings of 3D scenes, as they exhibit characteristics quite different from what's in its training set.

5 Conclusion

We have shown that we can perform 3D scene editing by implicitly maximizing a lower bound of the 3D scene's conditional log-likelihood using Score Jacobian Chaining with a latent diffusion model (e.g. InstructPix2Pix).

In section 4.3, we discussed some potential future directions to improve the quality of edited scenes. For example, improving the rendering quality of NeRF through better superresolution method would immediately alleviate the blurriness we see in our models. Moreover, fine-tuning InstructPix2Pix on a 3D model dataset would solve the failure-to-edit issue we see in Figure 2.

Finally, we would also like to explore a Mixture of Experts of approach. For example, we could average the score of InstructPix2Pix and Zero123 to try to obtain better estimate of the 3D model's log-likelihood.

References

- [1] Tim Brooks, Aleksander Holynski, and Alexei A. Efros. *InstructPix2Pix: Learning to Follow Image Editing Instructions*. 2023. arXiv: 2211.09800 [cs.CV].
- [2] Matt Deitke et al. *Objaverse: A Universe of Annotated 3D Objects*. 2022. arXiv: 2212.08051 [cs.CV].
- [3] Hiromichi Kamata et al. *Instruct 3D-to-3D: Text Instruction Guided 3D-to-3D conversion*. 2023. arXiv: 2303.15780 [cs.CV].
- [4] Ruoshi Liu et al. *Zero-1-to-3: Zero-shot One Image to 3D Object*. 2023. arXiv: 2303.11328 [cs.CV].
- [5] Ben Mildenhall et al. *NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis*. 2020. arXiv: 2003.08934 [cs.CV].
- [6] Anish Mittal, Anush Krishna Moorthy, and Alan Conrad Bovik. "No-Reference Image Quality Assessment in the Spatial Domain". In: *IEEE Transactions on Image Processing* 21.12 (2012), pp. 4695–4708. DOI: 10.1109/TIP.2012.2214050.

- [7] Ben Poole et al. *DreamFusion: Text-to-3D using 2D Diffusion*. 2022. arXiv: 2209.14988 [cs.CV].
- [8] Alec Radford et al. *Learning Transferable Visual Models From Natural Language Supervision*. 2021. arXiv: 2103.00020 [cs.CV].
- [9] Haochen Wang et al. *Score Jacobian Chaining: Lifting Pretrained 2D Diffusion Models for 3D Generation*. 2022. arXiv: 2212.00774 [cs.CV].